

Submission Worksheet

IT202-450-M2026 / module-03-worksheet-html-css-js-challenges

Submission Data

Course	IT202-450-M2026
Assignment	Module 03 Worksheet: HTML, CSS, JS Challenges
Student	Michos C. (mrc82)
Status	submitted
Worksheet Progress	100+%
Potential Grade	NaN/NaN (NaN%)
Started	[object Object]
Updated	6/22/2026, 6:34:43 PM
Grading Link	https://courses.ethereallab.app/assignment/v3/IT202-450-M2026/module-03-worksheet-html-css-js-challenges/grading/mrc82
View Link	https://courses.ethereallab.app/assignment/v3/IT202-450-M2026/module-03-worksheet-html-css-js-challenges/mrc82

Instructions

Complete the Module 3 layout and interaction challenges in the homework branch, then collect focused evidence for each challenge. It's best to solve and test each page first, then fill out the worksheet; however, you may also collect evidence as each challenge is solved.

Assignment Setup

- Starter reference: [Module 3 Homework starter folder](#)
- Required branch: `Module03-Homework`
- Required starting branch: `qa`
- Required homework folder: `public_html/m03/hw/`
- Starter files to copy: `index.html`, `challenge1.html`, `challenge2.html`, `challenge3.html`, `styles.css`, and `utils.js`.
- Recommended order:
 - Read the whole worksheet.
 - Copy the starter files; commit the baseline.
 - Add UCID/date and planning comments where edits are allowed; commit those comments.
 - Solve each challenge, test through `qa`, then collect the worksheet evidence.

Git Workflow Checklist

1. Check out `qa`.
2. Pull the latest `qa` branch with `git pull origin qa`.
3. Create the homework branch with `git checkout -b Module03-Homework`.
4. Copy the starter files into the Module 3 homework folder.
5. Commit the baseline starter files before solving the challenges.
6. Push the branch with `git push origin Module03-Homework`.
7. Open a pull request from `Module03-Homework` into `qa` and keep it open while you work.
8. Add planning comments before the final code for every challenge.
9. Commit planning comments before the finished solution for each challenge.

7. Open a pull request from `Module03-Homework` into `qa` and keep it open while you work.
8. Add planning comments before the final code for every challenge.
9. Commit planning comments before the finished solution for each challenge.
10. Commit working code in small pieces as each challenge starts working.
11. After all challenges pass locally, merge the homework pull request into `qa`.
12. Gather evidence from the deployed `qa` site and fill out the worksheet.
13. Export the worksheet PDF from the Learn Courses Platform.
14. Save the PDF in the repository under `docs/module03/`.
15. Add, commit, and push the PDF to the `Module03-Homework` branch.
 - `git add .`
 - `git commit -m "Add module 3 worksheet PDF"`
 - `git push origin Module03-Homework`
16. Upload the same PDF to Canvas.
17. Create and merge a pull request from `qa` into `prod` so the anticipated production URLs become live.
18. Switch back to `qa` with `git checkout qa`.
19. Pull the latest `qa` branch with `git pull origin qa`.

Shared Requirements

Apply these requirements to each challenge:

- Only edit code where the starter comments say edits are allowed.
- Add a comment with your UCID, date, and a brief summary of the solution approach.
- Add planning comments before the solution code.
- Commit the planning-comment stage before the final solution.
- Update the visible UCID in the page header where the starter asks for it.
- Test the file through the supported server path before collecting output evidence. For deployed evidence, use the working `qa` URL after the site has deployed.
- Use your own code, output, branch, pull request, and deployed URLs for evidence.

Challenge Evidence Format

Each challenge uses the same evidence pattern:

- Images:
 - Show the relevant code and the matching browser output.
 - Make sure the UCID/date comment is visible in the code evidence.
 - Make sure the deployed page URL is visible in the browser output evidence.
 - One combined screenshot is acceptable if the code and output are both readable.
 - Use two screenshots when one image would be hard to read.
- URLs:
 - Provide the direct GitHub file link from the homework branch.
 - Provide the anticipated production URL for the HTML file.
 - Capture the working `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
 - Use the tested Module 3 homework path, such as `/m03/hw/challenge1.html`, `/m03/hw/challenge2.html`, or `/m03/hw/challenge3.html`.
 - Each file URL must end in `.html`; zero credit for that URL entry if it does not.
- Response:
 - Explain the logic in your own words.
 - Describe the steps or decisions your CSS or JavaScript makes.
 - Do not restate the prompt or list the requirements.

Submission Check

Submission Check

- Confirm the homework branch has been pushed.
- Confirm the pull request into `qa` exists.
- Confirm the baseline, planning, and solution commits are visible.
- Confirm the `qa` version was reviewed before promotion.
- Confirm the anticipated production URLs use the same path as the tested `qa` URLs with `-qa` changed to `-prod`.
- Confirm the worksheet PDF is committed under `docs/module03/` on the homework branch.
- Upload the same worksheet PDF to Canvas.

Submission Progress

100%

Section #1: (3 pts.) Challenge 1 Fixed Header Content Footer

100%

- Goal: edit the allowed `style` area to solve the fixed-layout challenge.
- Expected layout:
 - The header stays fixed at the top of the page.
 - The footer stays fixed at the bottom of the page.
 - The middle content area scrolls independently.
 - The browser window itself should not show the main page scrollbar for the content.
 - The full page should fill the viewport height.
 - The borders around `header`, `main`, and `footer` should remain intact and visible.
- Workflow:
 - Update the visible UCID in the header tag.
 - Add UCID/date and planning comments before the solution code.
 - Commit the planning comments before committing the finished solution.
 - Test the HTML file locally before collecting evidence on `qa`.

Task #1 (3 pts.) – Challenge 1 Evidence

100%

Combo #1

Part 1: Images 100%

Details

- Show the relevant code with the UCID/date comment visible.
- Show the full browser output on the deployed `qa` page.
- Make sure the browser address bar is visible.



Challenge 1 code showing my UCID, date, and CSS layout.



Challenge 1 running on QA with the header, footer, and scrolling content visible

Part 2: URLs 100%

Details

- Paste the direct GitHub file link from the homework branch.

① Details

- Paste the direct GitHub file link from the homework branch.
 - The GitHub URL must end in `.html`; zero credit for this URL if it does not.
- Paste the anticipated production URL for the HTML file.
 - Capture the tested `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
 - The production URL must end in `.html`; zero credit for this URL if it does not.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Module03-Homework/public_html/m03/hw/challenge1.html



URL #2

<https://mrc82-it202-450-m2026-prod.onrender.com/m03/hw/challenge1.html>



✍ Part 3: Response 100%

① Details

- Explain how your CSS keeps the header and footer fixed.
- Explain how your layout allows only the content area to scroll.
- Explain how the page keeps the full viewport height.
- Do not restate the prompt.

Response

I made the page fill the whole screen and kept the header and footer in place. The middle section is the only part that scrolls, so the browser page itself does not move. This keeps the layout clean and makes sure the header and footer stay visible.

Supports **bold**, *italic*, [links](url), `code`, etc.

249 chars

Section #2: (3 pts.) Challenge 2 Header Content And Sidebars

100%

- Goal: edit the allowed `style` and `script` areas to solve the sidebar layout and toggle challenge.
- Expected CSS behavior:
 - The header is positioned at the top.
 - The content region takes up the remaining height.
 - The left and right sidebars dock to their respective sides.
 - Each sidebar uses 15% width when open.
 - Each collapsed sidebar collapses completely while keeping its button usable.
 - The content area uses the remaining width.
- Expected JavaScript behavior:
 - Each sidebar button has the appropriate event listener.
 - Each button toggles only its matching sidebar.
 - The toggle should not hide or lose access to the button.
 - The content area adjusts when the left sidebar, right sidebar, or both sidebars are collapsed.
- Workflow:
 - Update the visible UCID in the header tag.
 - Add UCID/date and planning comments before the solution code.
 - Commit the planning comments before committing the finished solution.
 - Test the HTML file locally before collecting evidence on `qa`.

📦 Task #1 (3 pts.) – Challenge 2 Evidence

100%

 Part 1: Images 100%

- Show the relevant CSS and JavaScript with the UCID/date comment visible.
- Show the full browser output on the deployed qa page.
- Make sure the browser address bar is visible.
- Include evidence that both sidebars can be toggled.

Challenge 2 JavaScript showing my UCID, date, and sidebar button logic.

Challenge 2 CSS showing my UCID, date, sidebar sizes, and collapsed layout.



Challenge 2 on QA with both sidebars open and the middle content between them.



Challenge 2 on QA with both sidebars collapsed and the middle content expanded.

- ① Details

- Paste the direct GitHub file link from the homework branch.
 - The GitHub URL must end in `.html`; zero credit for this URL if it does not.
- Paste the anticipated production URL for the HTML file.
 - Capture the tested `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
 - The production URL must end in `.html`; zero credit for this URL if it does not.



- Details**

- Explain how the sidebars and content area are sized.
- Explain how each button finds and toggles the correct sidebar.
- Explain how the content area changes when sidebars collapse or reopen.
- Do not restate the prompt.

Response

Response

I gave each open sidebar 15% of the page width, while the middle content uses the remaining space. Each button is connected to its own sidebar and adds or removes the collapsed class when clicked. When one or both sidebars close, the middle section automatically grows to fill the extra space, and the buttons stay available so the sidebars can be opened again.

Supports **bold**, *italic*, [links](url), ``code``, etc.

361 chars

Section #3: (3 pts.) Challenge 3 Carousel Layout With Swiping

100%

- Goal: edit the allowed **style** and **script** areas to solve the carousel challenge.
- Expected CSS behavior:
 - The header is positioned at the top.
 - The carousel takes up the full available width.
 - The buttons are centered.
 - The carousel panels fill the full height of the carousel.
 - The carousel content is centered vertically and horizontally.
 - The top navigation links remain page links, not carousel controls.
- Expected JavaScript behavior:
 - Each carousel button has the appropriate event listener.
 - The Previous and Next carousel buttons control the panels.
 - Only one panel is shown at a time.
 - The carousel moves through the panels in order.
 - The carousel loops when it reaches the first or last panel.
- Workflow:
 - Update the visible UCID in the header tag.
 - Add UCID/date and planning comments before the solution code.
 - Commit the planning comments before committing the finished solution.
 - Test the HTML file locally before collecting evidence on **qa**.

Task #1 (3 pts.) – Challenge 3 Evidence

100%

Combo #1

Part 1: Images 100%

Details

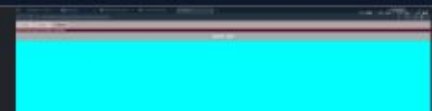
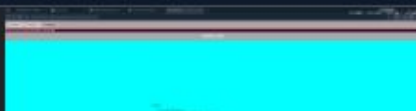
- Show the relevant CSS and JavaScript with the UCID/date comment visible.
- Show the full browser output on the deployed **qa** page.
- Make sure the browser address bar is visible.
- Include evidence that the carousel advances between panels.



Challenge 3 JavaScript showing my UCID, date, carousel controls, and swipe code.



Challenge 3 CSS showing my UCID, date, full-size carousel, and centered panels.





Challenge 3 on QA showing Panel 0 and the carousel controls.



Challenge 3 on QA showing the carousel moved to Panel 1.

Part 2: URLs 100%

Details

- Paste the direct GitHub file link from the homework branch.
 - The GitHub URL must end in `.html`; zero credit for this URL if it does not.
- Paste the anticipated production URL for the HTML file.
 - Capture the tested `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
 - The production URL must end in `.html`; zero credit for this URL if it does not.

URL #1

https://github.com/micopher/mrc82-IT202-450-M2026/blob/Module03-Homework/public_html/m03/hw/challenge3.html



URL #2

<https://mrc82-it202-450-m2026-prod.onrender.com/m03/hw/challenge3.html>



Part 3: Response 100%

Details

- Explain how your CSS sizes and centers the carousel.
- Explain how your JavaScript tracks the active panel.
- Explain how your logic loops at the first and last panel.
- Do not restate the prompt.

Response

I made the carousel fill the space below the header and centered the buttons and panel content. The JavaScript keeps track of the current panel number and shows only that panel. When it passes the last panel, it goes back to the first, and going backward from the first panel opens the last one.

Supports **bold**, *italic*, [links](url), ``code``, etc.

295 chars

Section #4: (+1 pt.) Challenge 3 Extra Credit Mouse Swipe

100%

- Goal: allow mouse swipe behavior on the carousel.
- Expected behavior: swiping with the mouse should cycle through panels in a way similar to the carousel buttons.
- URL note: this is part of Challenge 3, so it does not need a separate URL entry.
- Workflow:
 - Add UCID/date and planning comments before the solution code.
 - Commit the planning comments before committing the finished solution.
 - Test the HTML file locally before collecting evidence on `qa`.

Task #1 (+ 1 pt.) – Extra Credit Evidence

100%

Task #1 (+ 1 pt.) – Extra Credit Evidence

100%

Combo #1

Part 1: Images 100%

Details

- Show the relevant code with the UCID/date comment visible.



Mouse swipe code showing my UCID, date, and left/right swipe controls.

Part 2: Response 100%

Details

- Explain which mouse events your solution uses.
- Explain how your solution decides swipe direction.
- Explain how the swipe action reuses or matches the button carousel behavior.
- Do not restate the prompt.

Response

I used mousedown to save where the mouse drag starts and mouseup to check where it ends. The code compares the two positions to tell if the user dragged left or right. A left drag moves to the next panel, while a right drag moves to the previous panel. It uses the same next and previous functions as the buttons.

Supports **bold**, *italic*, [links](url), ``code``, etc.

314 chars

Section #5: (1 pt.) Misc

67%

Task #1 (0.33 pts.) – GitHub Details

100%

Details

- Show the repository evidence for the Module 03 homework.

Combo #1

Part 1: Images 100%

Details

- Screenshot the Commits tab of the pull request.
- The screenshot should show at least the baseline commit, planning-comment commits, and solution commits.



Commit	Author	Message	Files
1	micopher	Starter	1 file
2	micopher	Planning	1 file
3	micopher	Solution	1 file

Module 03 pull request showing the starter, planning, and solution commits.

Part 2: URLs 100%

Details

- Paste the pull request URL.
- The URL must end in `/pull/#`, where `#` is the pull request number; zero credit for this URL if it does not.

URL #1

<https://github.com/micopher/mrc82-IT202-450-M2026/pull/6> 🍏

Task #2 (0.33 pts.) – WakaTime Activity

100%

Details

- Visit the WakaTime dashboard.
- Open **Projects**.
- Find your repository.
- Screenshot the overall time near the repository name (top of page).
- Screenshot the file-level activity (near the bottom).
- The purpose is evidence of activity; the duration isn't tied to the grade.
- The visual graphs are not required.



Task #3 (0.33 pts.) – Reflection

Details

- Answer each question with at least a few reasonable sentences.

Task #1 (0.11 pts.) – What did you learn during this assignment?

100%

Response

100%

Response

I learned how to use CSS and JavaScript together. I also learned how to make sidebars and a carousel work.

Supports **bold**, *italic*, [links](url), `code`, etc.

106 chars

Task #2 (0.11 pts.) – What was the easiest part of the assignment?

100%

Response

I would say the Git and pull request steps were the easiest. They were similar to the previous modules, so I already know what to do.

Supports **bold**, *italic*, [links](url), `code`, etc.

133 chars

 Saving...

Task #3 (0.11 pts.) – What was the hardest part of the assignment?

100%

Response

The coding was the hardest part. Getting the layouts and buttons to work correctly took the most time to be honest.

Supports **bold**, *italic*, [links](url), `code`, etc.

115 chars

 Saving...

End of Assignment - submission has been recorded.